Kubernetes Self Healing and Rolling Updates

Nadezhda Osipova - CISC3010 Project 11 - Final report - 5 November 2026

Research question and main result

How do Kubernetes recovery and rollout settings affect ready capacity, completion time and HTTP failures? A replicated stateless service was observed during controlled Pod deletion and updates from v1 to v2. In the original 12 runs at 10 requests/s, all 9,600 requests succeeded. A retained more ready capacity than B, while update times were similar. A separate pressure series recorded fewer failures with A, but errors before updates limit the causal interpretation. Recovery of the desired replica count and continuity of client service are distinct measurements. A separate optional extension measures CPU-based autoscaling under changing load.

Architecture and implementation

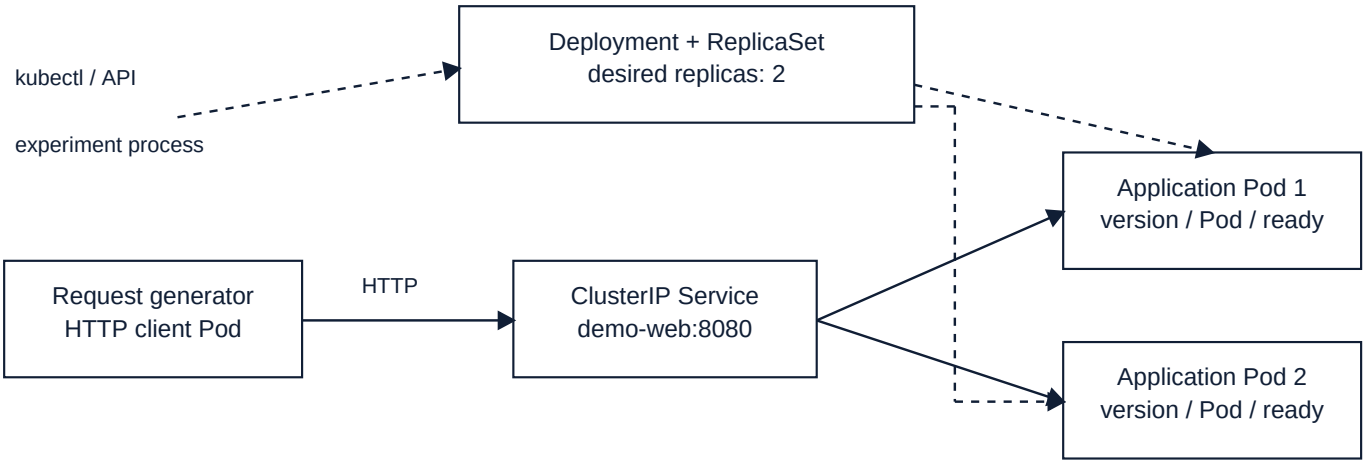

Figure 1. Original experiment: solid links show HTTP requests; dashed links show control paths. All containers share one kind node. The optional extension uses a separate Service and Deployment.

The client runs in its own Pod and sends requests through the ClusterIP Service `demo-web:8080`. Responses contain `version`, `pod` and `ready`. The experiment process uses kubectl and the Kubernetes API to trigger actions and sample Pods, Deployment and EndpointSlices. Deployment coordinates ReplicaSets; ReplicaSet maintains the requested Pod count. The scheduler assigns a node and kubelet starts containers and performs probes [3,4].

The application uses Python's ThreadingHTTPServer [7]. v1 and v2 use the same logic and differ only in their version label. The application models an 8 s startup, returning HTTP 503 on `/` and `/ready` until prepared. Readiness runs every 1 s. A Ready Pod can receive Service traffic immediately; `minReadySeconds: 2` requires continuous readiness before Deployment counts it as Available. Readiness failure alone does not restart a container [4,5].

Each application Pod requests 100m CPU / 64 MiB and has limits of 250m CPU / 128 MiB. Graceful termination includes a 5 s preStop pause within a 15 s grace period. This is controlled deletion with time to stop, rather than a sudden node outage. The app has no database or persistent user state. Project-specific code consists of the HTTP app, manifests, workload generators, experiment runners, assertions and analysis; Kubernetes supplies the actual controllers.

Experimental design and recorded environment

Measurements used Ubuntu 24.04 under WSL2 and one kind node on a shared ASUS Vivobook. The host has 16 logical CPUs and about 16 GB RAM; saved node data reports 16 CPUs and 7,729,228 KiB memory. Saved Kubernetes/kubectl versions are 1.37.0 and node containerd is 2.3.4. Final validation used kind 0.33.0 and Python 3.12.3. These are shared virtual resources. The application image uses `python:3.12-slim`; mutable image tags and background load limit bit-for-bit numerical reproduction [6].

The original series contains 3 repeats of baseline, deletion, A and B. Each run first restores two stable Ready v1 Pods, then sends 10 requests/s for 80 s (800 attempts), using a threaded client with up to 16 workers and a 1 s timeout. The intervention starts near 15 s. A uses maxUnavailable=0 / maxSurge=1; B uses 1 / 0. Order alternates A/B, B/A, A/B across repeats. Per-Pod limits and offered workload stay fixed. A intentionally permits extra resources; equal limits per Pod do not imply equal total resource use.

Original results

Condition	Failed / sent	Mean completion s (range)	Mean run p95 ms	Ready range
Baseline	0 / 2,400	n/a	31.99	2-2
Delete	0 / 2,400	12.14 (11.44-12.49)	30.88	1-2
A	0 / 2,400	25.46 (25.44-25.50)	29.43	2-3
B	0 / 2,400	24.80 (24.45-25.43)	28.68	1-2

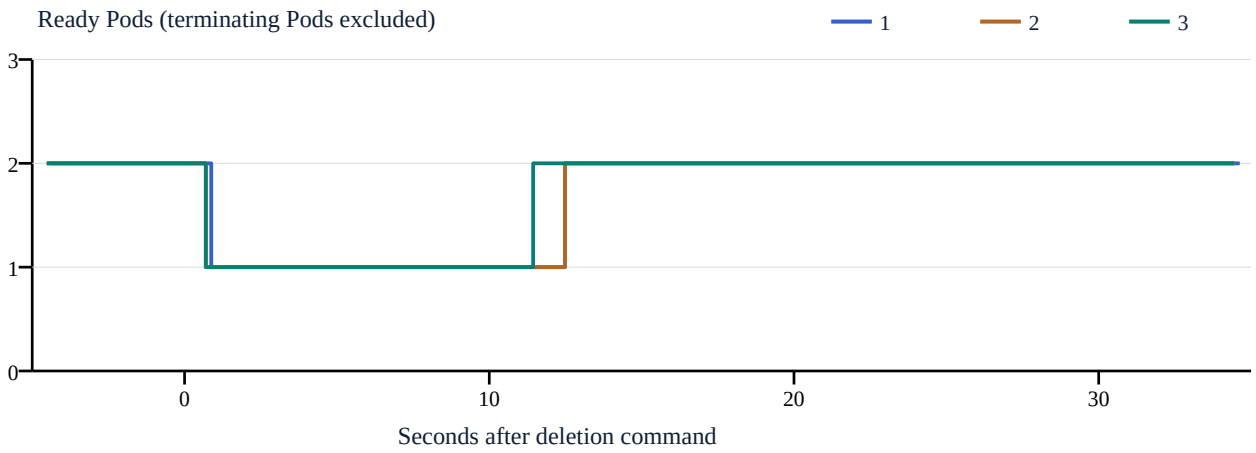

Figure 2. All 3 original deletion runs. Ready excludes terminating Pods. Time zero is the deletion command. Lines connect discrete observations as steps.

Deletion temporarily reduced Ready capacity to 1; a new UID then became Ready. The surviving copy continued serving requests. A retained at least 2 Ready copies during the original updates; B sometimes relied on 1. The 0.66 s mean timing difference is comparable to roughly 1 s polling and does not establish a reliable speed advantage. Grouped p95 is the mean of 3 per-run successful-request p95 values, not a pooled percentile. Timing ranges show repeat variation, not confidence intervals. Three later deletion demos remain separate from this table.

Separate pressure experiment

Preliminary pilots exposed late dispatch in the threaded pressure client. Those records and diagnostics remain included, but final pressure comparisons use one async HTTP client throughout. Independent checks at 250 requests/s lasted 20 s (5,000 attempts), repeated 3 times with 1 and 2 copies in alternating order. One-copy failure rates were 2.72%, 2.48% and 1.58%; two-copy checks had no failures. This selected a workload at which losing a ready copy might affect clients. It is not a universal capacity estimate.

The subsequent update series uses 250 requests/s for 60 s (15,000 attempts), with 3 repeats per strategy and alternating order. Both still target 2 replicas and retain the original per-app limits. The separate generator has a 2 CPU / 512 MiB limit, up to 512 concurrent requests and a 1 s timeout. All 120,000 attempts across the 6 capacity checks and 6 updates were accounted for; the selected series passed generator saturation and schedule-delay checks.

Strategy	Failed / sent	Whole-run failures	Mean run successful p95
A	370 / 45,000	0.82%	1.90 ms
B	2,595 / 45,000	5.77%	21.23 ms

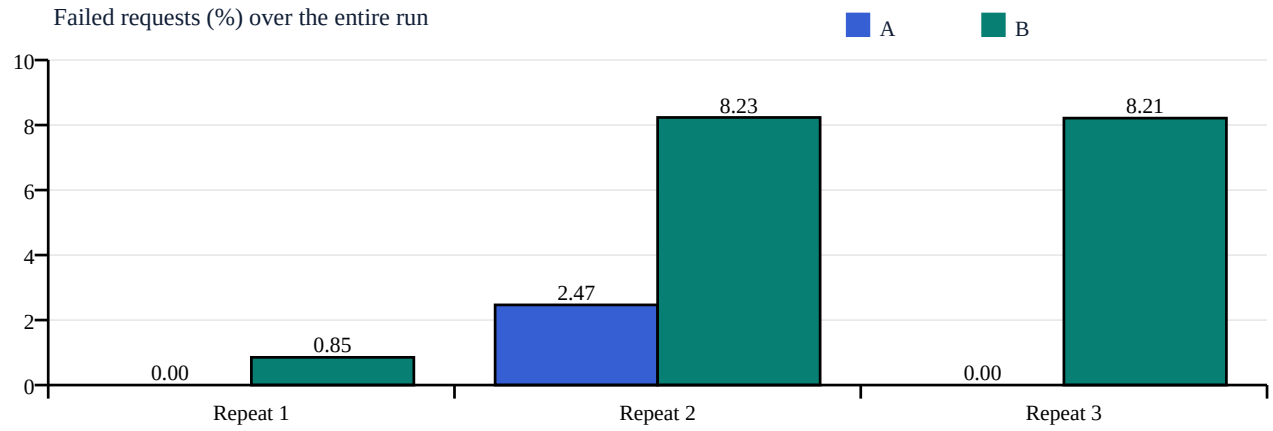

Figure 3. Each bar includes the full 60 s run. These totals include pre-update errors and are not estimates of update-only failures.

The starting conditions varied. Among 3,500 attempts scheduled before client second 14, A repeat 2 had 135 failures and B repeat 2 had 15; the other repeats had none in this early window. The update starts near second 15. These earlier errors were not subtracted, and all 6 rollout runs remain included. A had fewer recorded failures in this series, but the full difference cannot be attributed solely to the rollout settings.

In B repeat 3, the remaining old Pod lost readiness while v2 was starting and observed Ready count reached 0. maxUnavailable limits planned disruption; an independent readiness failure can reduce availability further. Successful-request p95 excludes timeouts, so low p95 does not mean every attempt was fast. Different clients prevent a clean causal comparison of absolute latency between the original and pressure series. Pressure details, pilots and CPU/scheduling diagnostics are retained in `experiments/pressure/`.

Correctness evidence and measurement limits

The audit recomputes counts/p95, checks request sequences, serving Pod names, replacement UIDs and final v2/Available states. The original rollouts' final 100 scheduled successful responses use v2. Raw records and checksums are included; polling is not lossless.

Recovery ends at the first observed Ready replacement; rollout completion requires 2 Ready non-terminating v2 Pods and 2 updated Available replicas. Monotonic durations include API/polling delay. Calendar-clock shifts invalidate the original additional UTC-aligned 45 s window, which is excluded.

Three repeats on one shared host do not establish machine-failure tolerance. Failed pilots and a pre-workload setup timeout remain included. The verification record is in `artifacts/verification.json`.

Optional horizontal Pod autoscaling

The same v1 app runs in separate `demo-autoscale`, retaining per-Pod resources. Metrics Server v0.9.1 supplies CPU measurements; HPA controls 2-6 replicas [10,11]. The target is 60% of the 100m CPU request, not the 250m limit; downscale stabilization is 60 s (default 300 s). Each of 3 low/high/low cycles uses 10 requests/s for 60 s, 400 requests/s for 150 s, then 10 requests/s for 150 s. Sampling is approximately every 5 s; phase boundaries use client-start receipt.

Repeat	Peak Ready	First >2 Ready after rise (s)	Back to 2 after fall (s)
1	6	50.93	80.91
2	6	40.92	90.97
3	6	35.91	85.92

All 3 cycles reached 6 Ready copies and returned to 2. Timings include sampling delay. All 186,300 attempts were accounted for: 31/180,000 high-load requests timed out; low-load failures and generator errors were 0. CPU remained above target at the 6-copy cap; ScalingLimited was recorded. Without a fixed-size control at 400 requests/s, this shows bounded scaling response, not a latency benefit.

Connection to Borg and changes after presentation

Borg is Google's internal cluster manager. Kubernetes is a separate open-source system developed using lessons from Borg/Omega, with overlapping contributors [1,2]. Borg section 2.3 describes gradual updates with disruption limits; section 4 discusses availability. This project applies those ideas through replicas, replacement and update budgets. Kubernetes documentation supplies the actual Ready/Available and A/B semantics [3-5]. The experiment simplifies away Borg's scale, scheduling policies and master architecture.

After presentation questions, the reading connection was clarified. Other additions are executable setup, HTTP tests, independent raw-data audits, reproducible figures, English documentation and the separate autoscaling extension.

AI assistance

I used OpenAI Codex to help develop and document the project. I ran the original experiments, and Codex ran the pressure and autoscaling extensions. Raw logs, automated tests and live cluster checks were used to verify the results.

References

[1] A. Verma et al. Large-scale Cluster Management at Google with Borg. EuroSys 2015. Main reading, especially sections 2.3 and 4. https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/

[2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes. Borg, Omega, and Kubernetes. ACM Queue 14, 2016. https://research.google/pubs/borg-omega-and-kubernetes/

[3] Kubernetes. Kubernetes Self-Healing. https://kubernetes.io/docs/concepts/architecture/self-healing/

[4] Kubernetes. Deployments, including rolling updates, minReadySeconds and terminating Pods. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[5] Kubernetes. Configure Liveness, Readiness and Startup Probes. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

[6] Kubernetes SIGs. kind Quick Start. https://kind.sigs.k8s.io/docs/user/quick-start/

[7] Python Software Foundation. http.server and unittest documentation. https://docs.python.org/3.12/library/http.server.html and https://docs.python.org/3.12/library/unittest.html

[8] Docker. Dockerfile reference and Docker Desktop WSL2 backend. https://docs.docker.com/reference/dockerfile/ and https://docs.docker.com/desktop/features/wsl/

[9] ReportLab. ReportLab PDF Toolkit documentation. Used only for reproducible figures and PDF formatting. https://docs.reportlab.com/

[10] Kubernetes. Horizontal Pod Autoscaling: CPU requests, control loop and stabilization. https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[11] Kubernetes SIGs. Metrics Server v0.9.1, compatibility, requirements and Apache-2.0 upstream manifest. https://github.com/kubernetes-sigs/metrics-server/releases/tag/v0.9.1 and https://github.com/kubernetes-sigs/metrics-server

Documentation checked 5 November 2026. Numeric evidence comes from the submitted raw records, not the readings. Original and pressure cohorts are fixed in `experiments/run-manifest.json`; autoscaling has its own manifest and `artifacts/autoscaling-results.json`. `artifacts/runs.csv`, `key-results.json` and `data-checksums.json` provide the audit trail. The README gives commands to regenerate the result tables and vector figures.